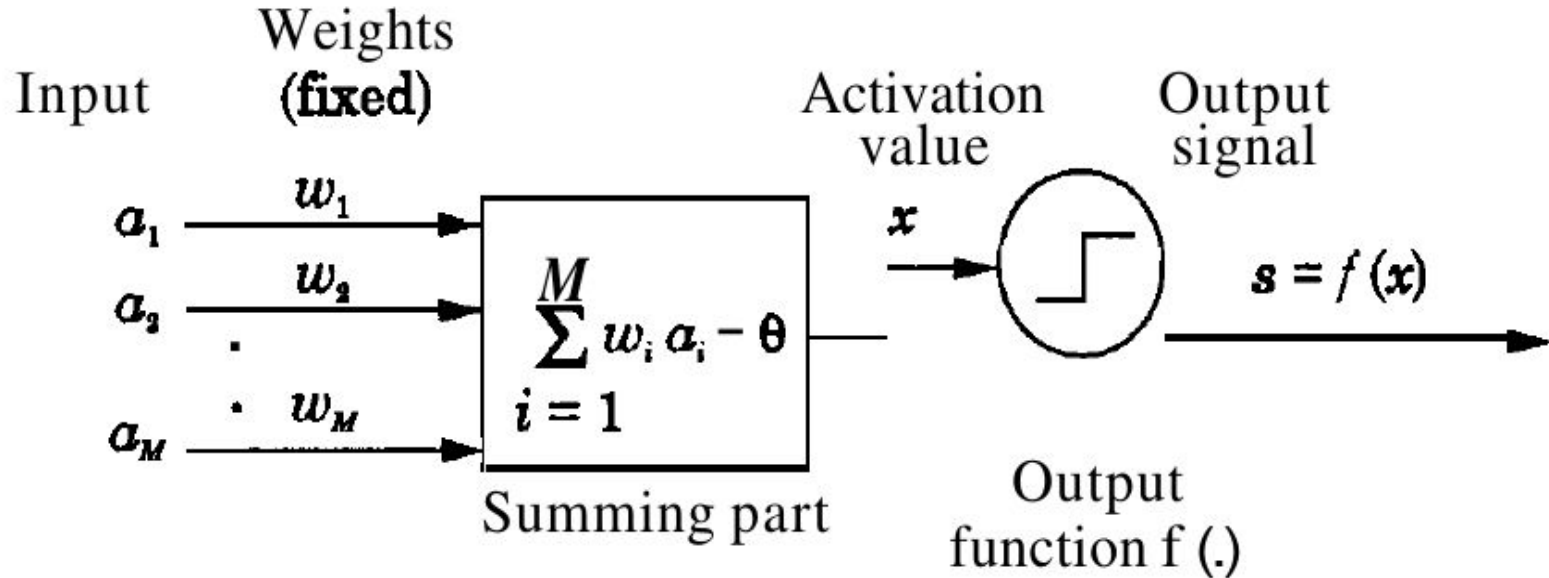


ANN and Backpropagation

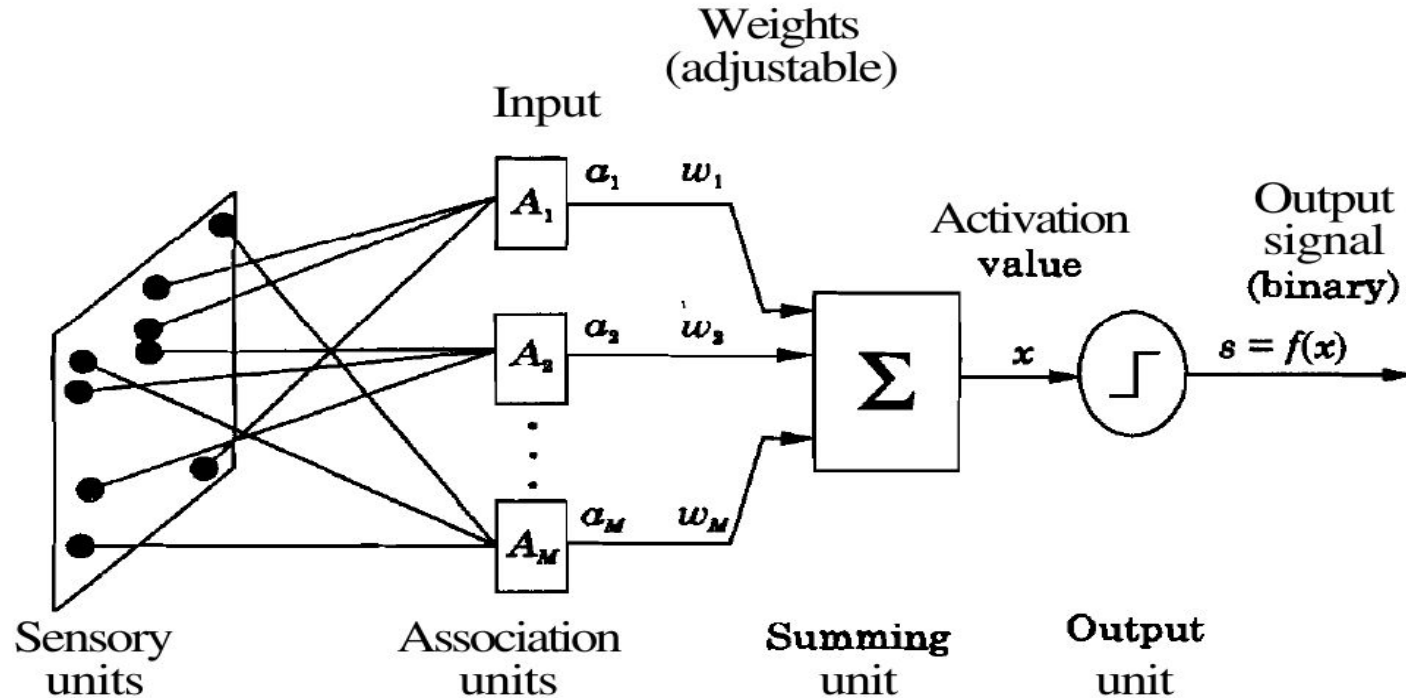
Initial Neural Models

McCulloch and Pitt's Model



Mcculloch and Pitt's Model (Figure taken from [1])

Rosenblatt's Model



Rosenblatt's Model (Figure taken from [1])

Initial Neural Models

- Both were tested on logic gates
- The basic neural network can be modeled as a composition of functions
 - For inputs x_1, x_2 , the output $y = g(f(x_1, x_2))$

$$f = \sum_{i=1}^2 w_i x_i - \theta = \sum_{i=0}^2 w_i x_i$$

where $w_0 = -\theta$ and $x_0=1$

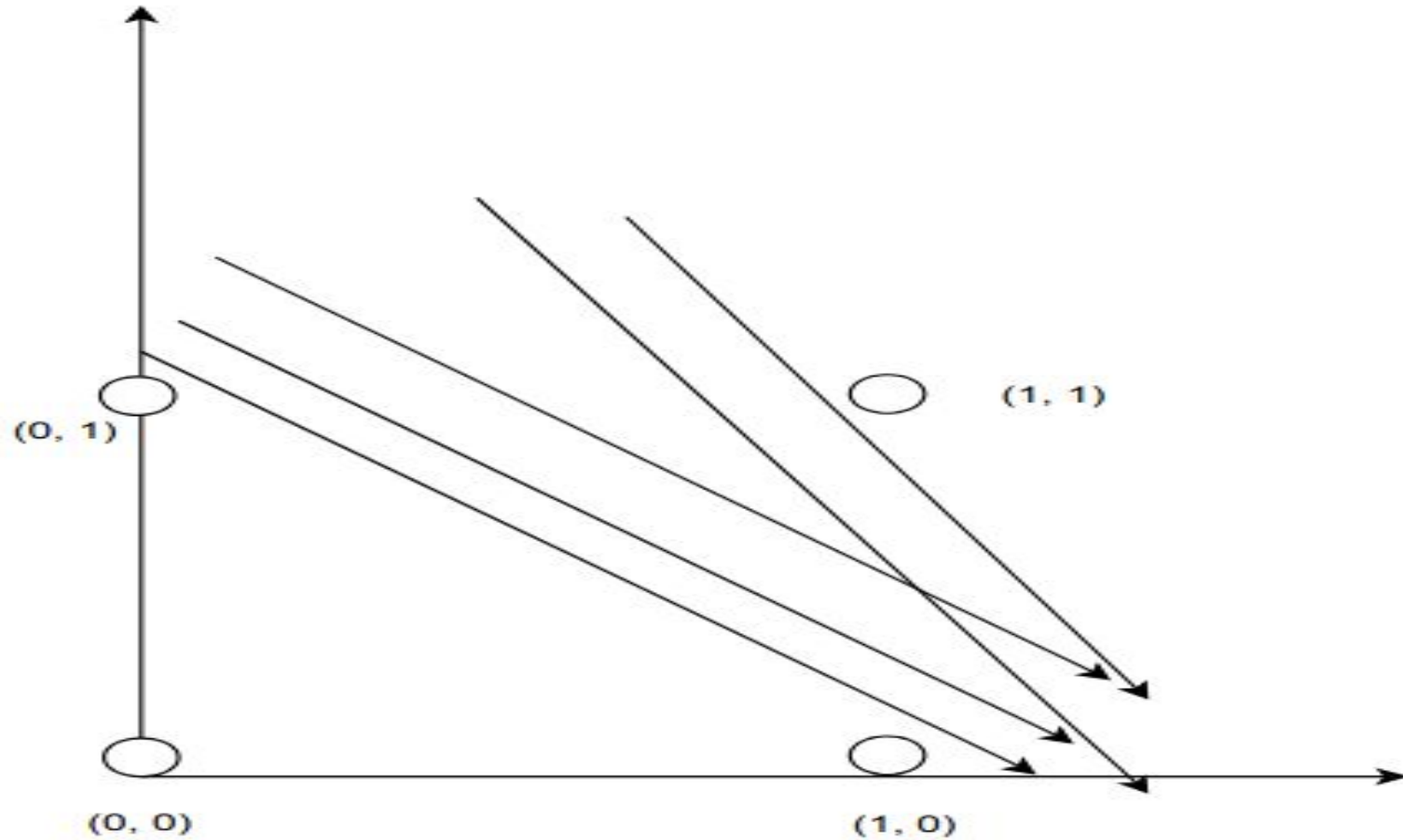
- $g(x) = 1$ if $x > 0$ else 0
- The first model for neurons was proposed by McCulloch and Pitts in 1943
 - Where the weights are fixed
- Single Layer Perceptron was 1st suggested by Frank Rosenblatt in 1958
 - An update rule to change the weights was also proposed

Example Using AND Operation

<u>x1</u>	<u>x2</u>	<u>y (AND)</u>
0	0	0
0	1	0
1	0	0
1	1	1

If we design a neural model for these using McCulloch and Pitt's model with fixed weights, $y = 1$ when $w_0 + w_1x_1 + w_2x_2 \geq 0$ else 0
 $w_0 = -2, w_1 = 1, w_2 = 1$ will satisfy this.
Similarly, $w_0 = -1.01, w_1 = 1, w_2 = 1$ will also satisfy this. The solutions are infinitely many.

Infinitely Many Lines



Fixed Weights

- Finding fixed weights is a hit and trial method
 - Find triplets of w_o , w_1 , w_2 so that all the conditions are satisfied
 - i. Find a triplet and check the conditions
 - ii. If condition is not satisfied, repeat step (i)
 - iii. If condition is satisfied, return the triplet
 - Is there any way to create a mechanism such that weights will be updated automatically?

Update/Delta Rule

- If the model's prediction and the actual output do not match
 - Update the weights
 - There are 2 cases
 - When model's prediction = 0 and the actual output = 1
 - Means $\sum w_i x_i < 0$, hence increase the weights
 - When model's prediction = 1 and the actual output = 0
 - Means $\sum w_i x_i \geq 0$, hence reduce the weights
- If the model's prediction and the actual output match
 - Do nothing

Mathematical Formulation of Update Rule

1. Let $x = [x_0, x_1, x_2]$ (where $x_0=1$), $w = [w_0, w_1, w_2]$, y be the inputs, weights, and the outputs
2. Randomly initialize w
3. When $\sum w_i x_i < 0$ and $y = 1$
 - a. Increase the weights
 - i. $w = w + x$
4. When $\sum w_i x_i \geq 0$ and $y = 0$
 - a. Decrease the weights
 - i. $w = w - x$
5. Run 3 and 4 for N number of times.
 - a. Another way to stop is when the weights in two successive steps do not change

A generic formulation for the update is:

$$w_{t+1} = w_t + \Delta w_t$$

$\Delta w_t = \eta * (y - g) * x$, where $g = 1$ when $\sum w_i x_i \geq 0$ else 0

η is a learning parameter and $\eta < 1$

Criticism from Minsky and Papert

- Rosenblatt claimed that any classification task can be solved by perceptrons
- Minsky and Papert in their book “Perceptrons: An Introduction to Computational Geometry” challenged this claim and showed that the perceptron model is not capable of solving the **XOR** operation between two inputs
 - You can not find any combination of w_0 , w_1 , w_2 that can solve the XOR problem shown in the earlier slides
- After this, the interest in Neural Networks subsided
- In 1980s, multi layered neural networks with automatic weight updates via backpropagation were proposed
 - In 1986, David Rumelhart invented back-propagation networks
 - These back-propagation networks are *slow-learners*, needs lots of iterations to learn

Multi Layered/Feed Forward Neural Network

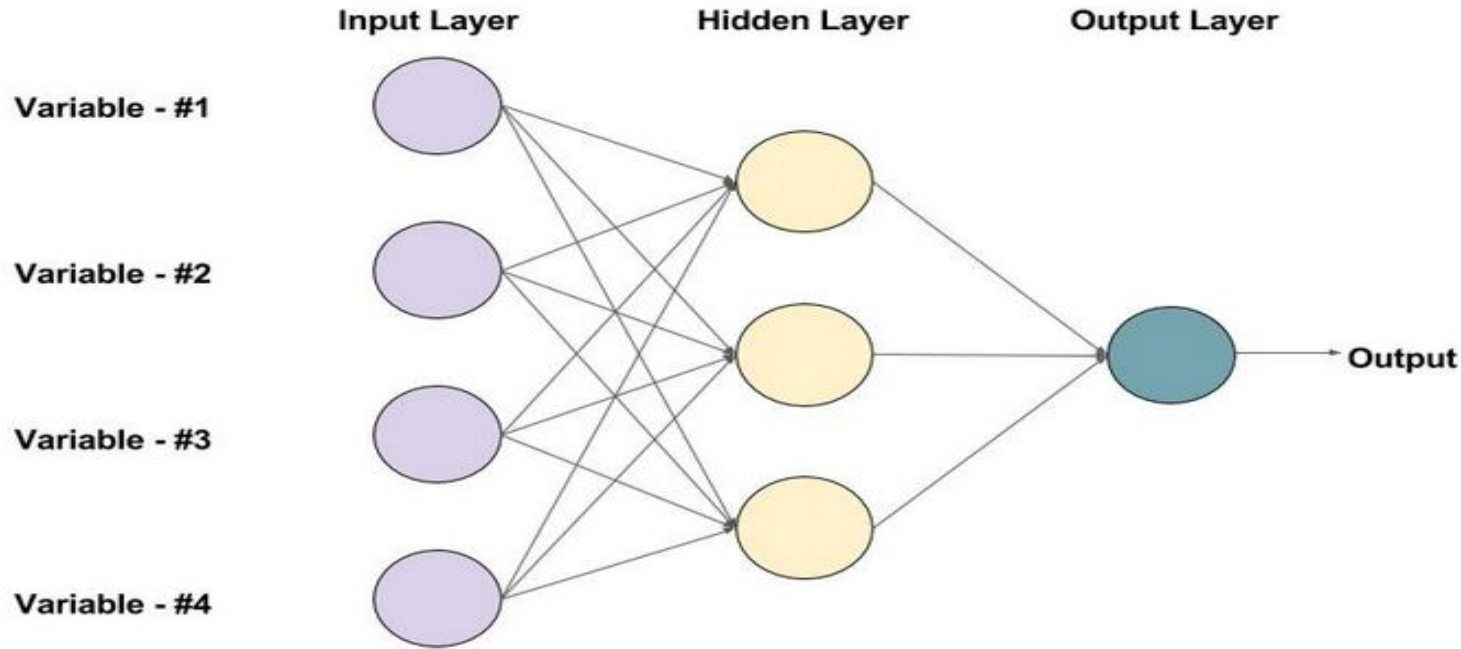
- Neural networks infer rules or regularities in patterns from thousands of training examples.
- Example: Shapes of different digits in the handwritten digit classification task (MNIST dataset)



MNIST Dataset Example (image credits:

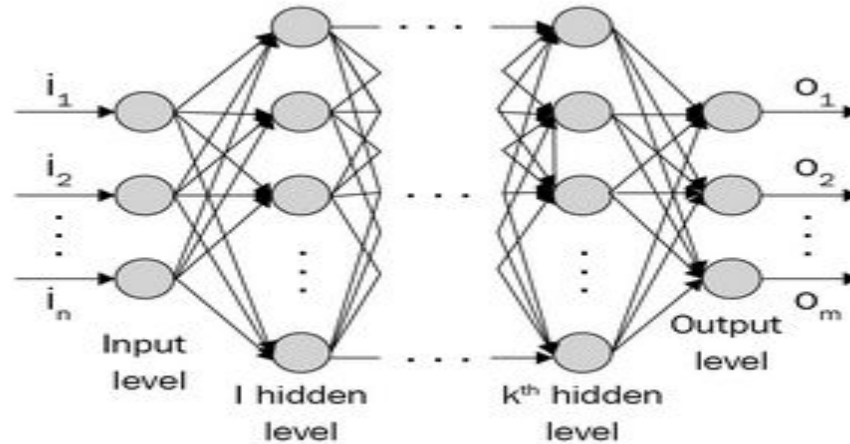
<http://neuralnetworksanddeeplearning.com/chap1.html>)

- A neural network is a complex connection between multiple neurons in multiple layers.



An example of a Feed-forward Neural Network with one hidden layer (with 3 neurons)

A More Complex Feed Forward Neural Network

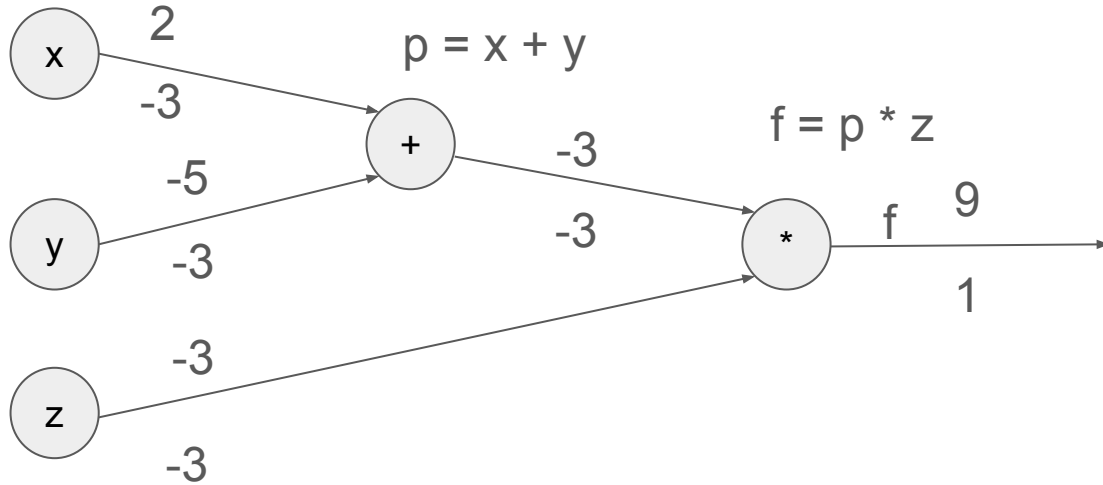


A Feed Forward Neural Network with Multiple Hidden Layers

How to Train a Feed Forward Neural Network?

- To train a complex feed forward neural network with multiple hidden layers, we need to have a more robust and stable update rule than the update rule suggested by Rosenblatt
- Let us now look at an interesting concept that is the foundation of neural network
 - Computational Graphs
 - Forward Pass
 - Backward Pass (Backpropagation)

A Simple Computational Graph



Example graph taken from the CS231n - Lecture 4 Slides by Andrej Karpathy

Workings in a Computational Graph

Forward Pass

Given the inputs, the values are computed at different computational units or gates (in the example: + and * gates)

Backward Pass

After the forward pass has been completed, the gradients are computed based on the final output (f in the example) and successive backward gradients are computed that take into account the gradient a gate receives and its local gradient using chain rule of derivatives. The gradients propagate through every computational unit in your graph until you reach the inputs.

Backward Pass

$$p = x + y \quad f = p' * z$$

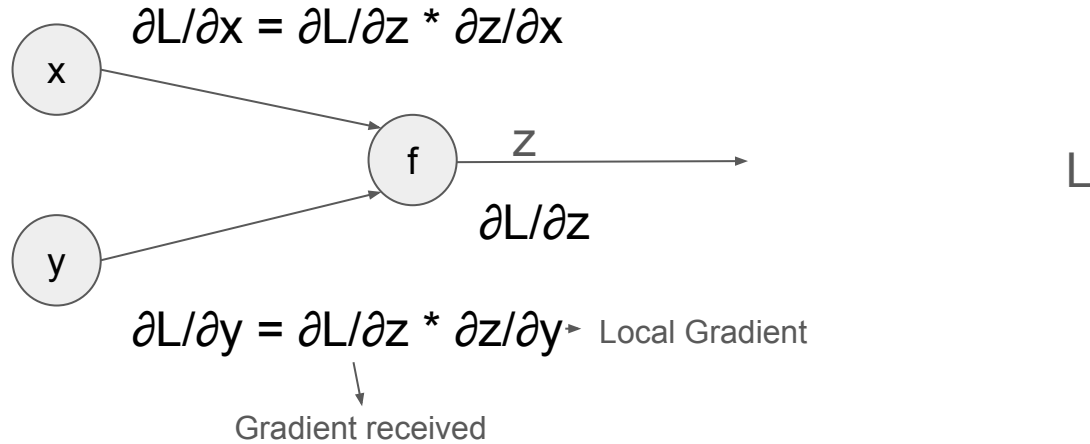
$$\partial f / \partial f = 1 \quad \partial f / \partial p = z \quad \partial f / \partial z = p$$

$$\partial f / \partial x = \partial f / \partial p * \partial p / \partial x \quad \partial p / \partial x = 1 \text{ [local gradient]}$$

$$\partial f / \partial y = \partial f / \partial p * \partial p / \partial y \quad \partial p / \partial y = 1 \text{ [local gradient]}$$

We are interested in finding how the output changes based on the changes of the inputs x , y , z which are captured in the gradients of $\partial f / \partial x$, $\partial f / \partial y$, $\partial f / \partial z$

Major Takeaway

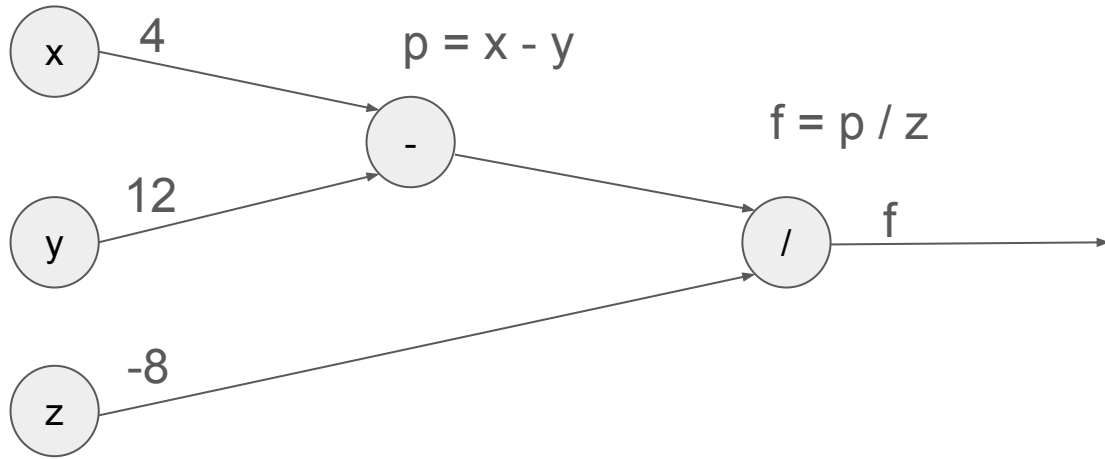


Sigmoid Gate (Interesting Function)

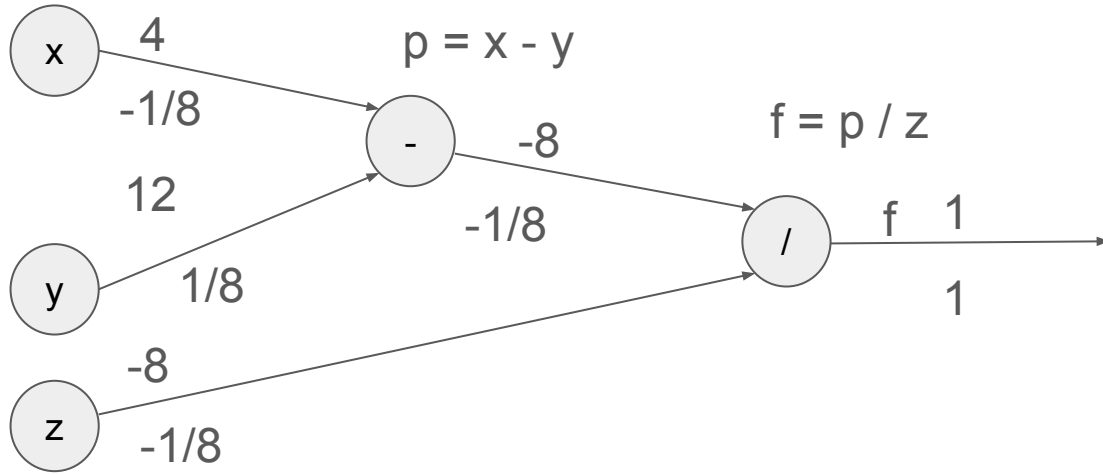
$$\sigma(x) = 1 / (1 + e^{-x}) = e^x / 1 + e^x$$

$$\begin{aligned} d\sigma(x)/dx &= e^{2x} / (1 + e^x)(1 + e^x) = (e^x / 1 + e^x) * (1/ 1 + e^x) \\ &= \sigma(x) (1 - \sigma(x)) \end{aligned}$$

Question 1



Question 1

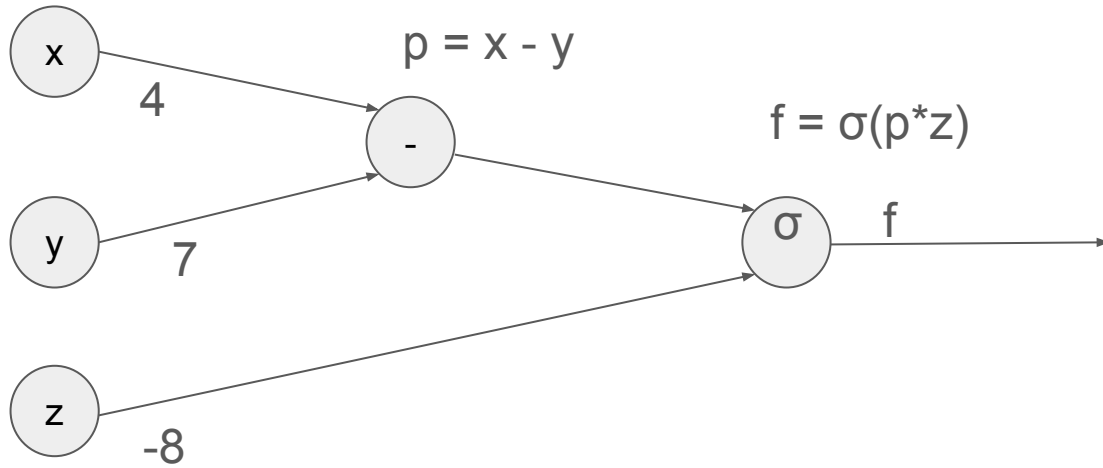


$$\frac{\partial f}{\partial p} = 1/z \quad \frac{\partial f}{\partial z} = -p/z^2$$

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial p} * \frac{\partial p}{\partial x} \quad \frac{\partial p}{\partial x} = 1 \text{ [local gradient]}$$

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial p} * \frac{\partial p}{\partial y} \quad \frac{\partial p}{\partial y} = -1 \text{ [local gradient]}$$

Question 2



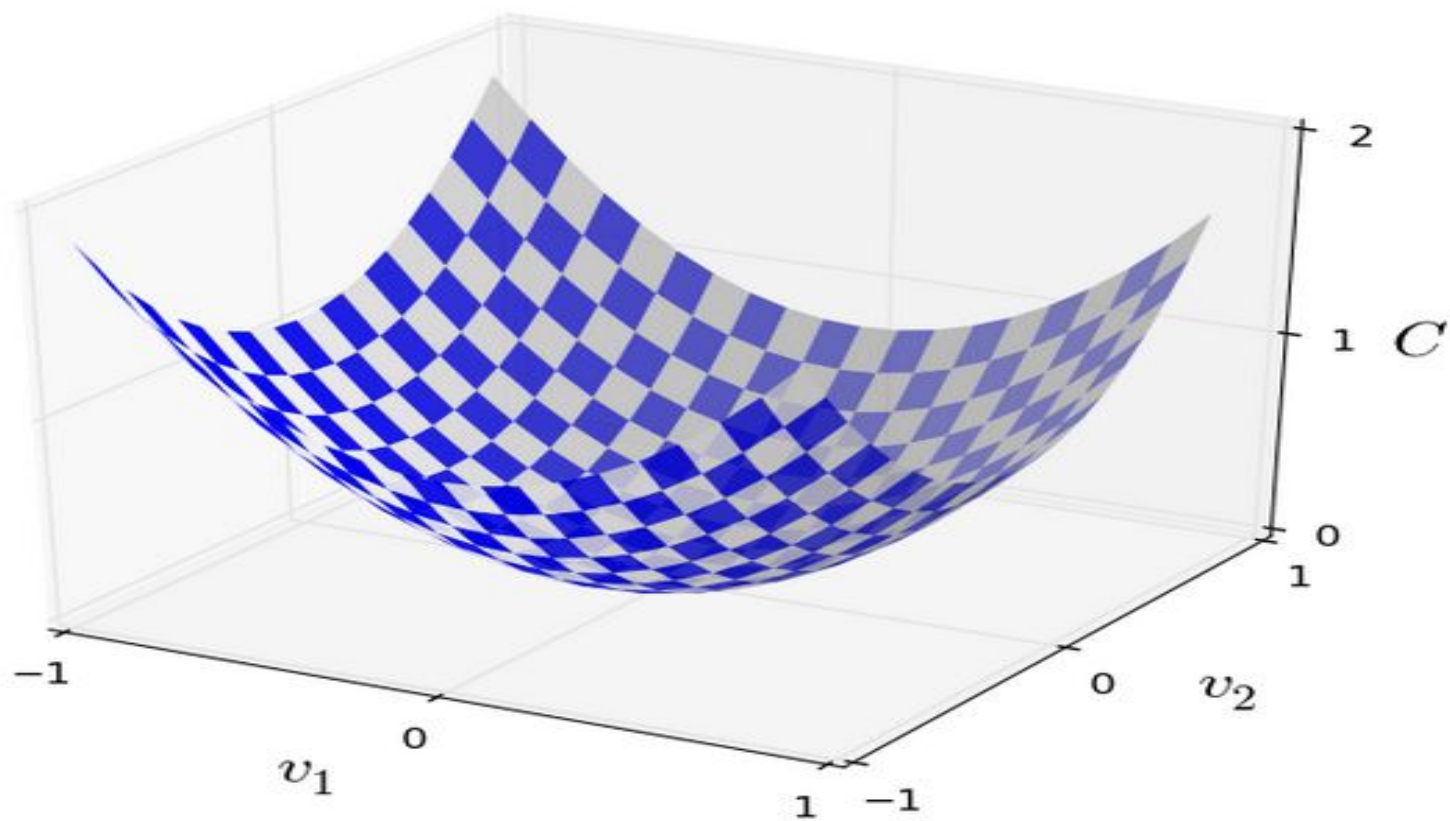
Training a Neural Network

- After all the interesting stuff using computational graphs, let us design a neural network with backpropagation
- The following computations will be used:
 - $\text{Error} = \text{Actual Target} - \text{Computed Target (in forward pass)}$
 - $\Delta w = \text{learning rate} * \text{local gradient} * \text{gradient received (from error term)}$
 - $w_{i+1} = w_i + \Delta w_i$

Standard Loss Functions

- Two loss functions are widely used
 - Mean Squared Error (MSE)
 - $(y_{\text{target}} - y_{\text{pred}})^2$
 - Binary Cross Entropy Loss (Can be Multi Class)
 - $y_{\text{target}} \log y_{\text{pred}} + (1 - y_{\text{target}}) \log (1 - y_{\text{pred}})$
- For multiple training examples, we usually sum the error terms for each example

Loss Curve



Training Details

- MSE Loss for n training examples/samples for Feed Forward Network involving L I

$$C = \frac{1}{2n} \sum_x \|y(x) - a^L(x)\|^2,$$

- Binary Cross Entropy Loss with n training examples/samples

$$C = -\frac{1}{n} \sum_x [y \ln a + (1 - y) \ln(1 - a)],$$

- Components of Error gradient at the output layer δ^L

$$\delta_j^L = \frac{\partial C}{\partial a_j^L} \sigma'(z_j^L).$$

- An equation for the error δ^l in terms of the error in the next layer, δ^{l+1} : (where $\odot$ is called Hadamard or element wise product)

$$\delta^l = ((w^{l+1})^T \delta^{l+1}) \odot \sigma'(z^l),$$

- An equation for the rate of change of the cost with respect to any weight in the network:

$$\frac{\partial C}{\partial w_{jk}^l} = a_k^{l-1} \delta_j^l.$$

$$\frac{\partial C}{\partial w} = a_{\text{in}} \delta_{\text{out}},$$

$$\frac{\partial C}{\partial w} = a_{\text{in}} \times \delta_{\text{out}}$$


The diagram illustrates a single connection between two nodes, represented by circles. An arrow points from the left node to the right node. Above the arrow, the expression $a_{\text{in}} \times \delta_{\text{out}}$ is written, indicating the calculation of the error gradient for the weight w connecting these two nodes.

All the Images are in slides 26-29 from
<http://neuralnetworksanddeeplearning.com/chap2.html>

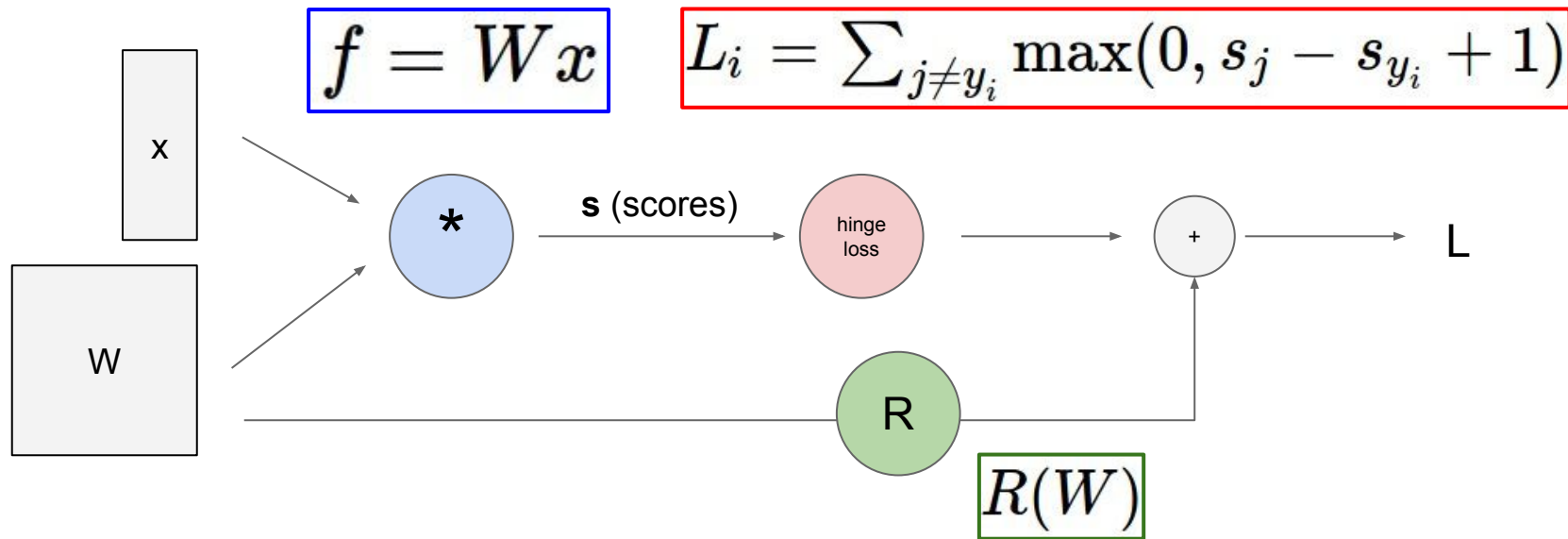
References

1. Artificial Neural Networks, B. Yegnanarayana, Prentice Hall of India
2. CS231n Winter 2016: Lecture 4: Backpropagation, Neural Networks 1 - <https://www.youtube.com/watch?v=i94OvYb6noo>
3. How the backpropagation algorithm works - <http://neuralnetworksanddeeplearning.com/chap2.html>

Lecture 4:

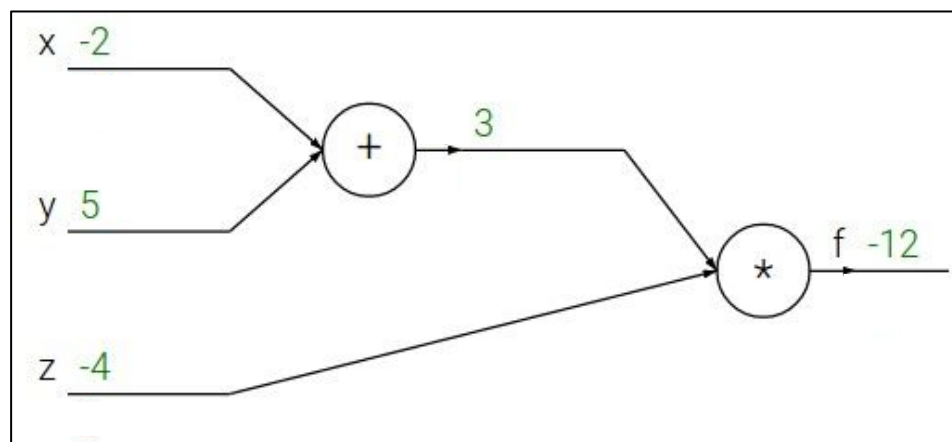
Backpropagation and Neural Networks part 1

Computational Graph



$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$



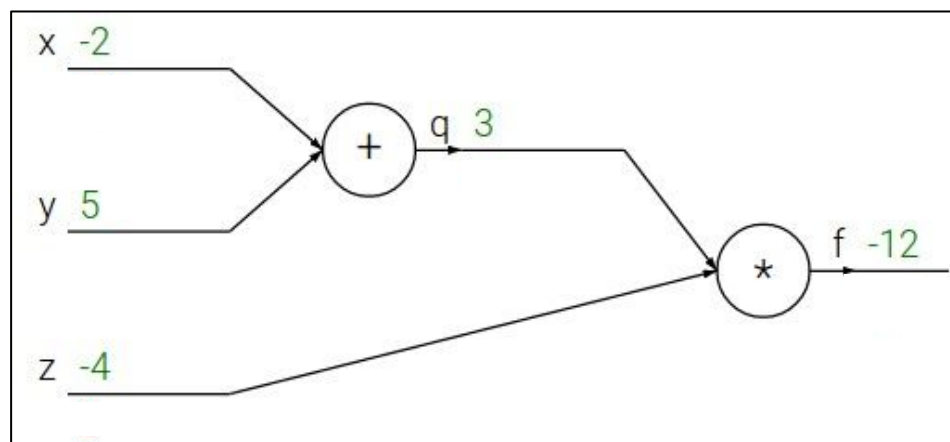
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



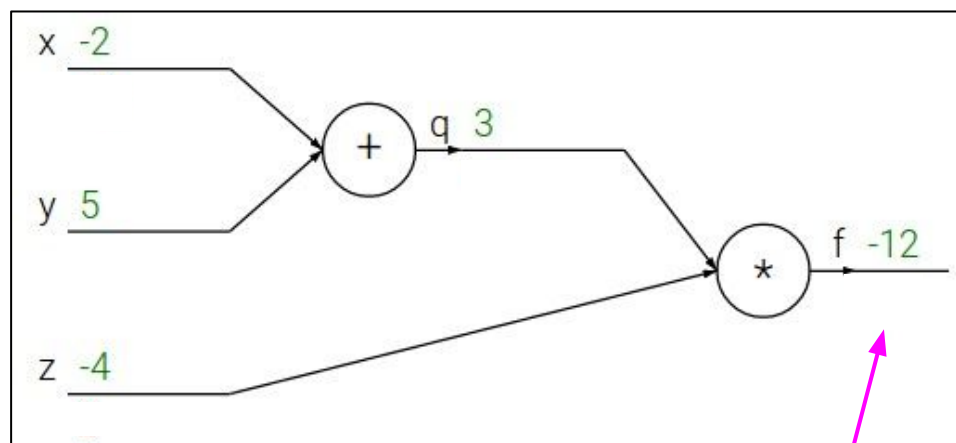
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial f}$$

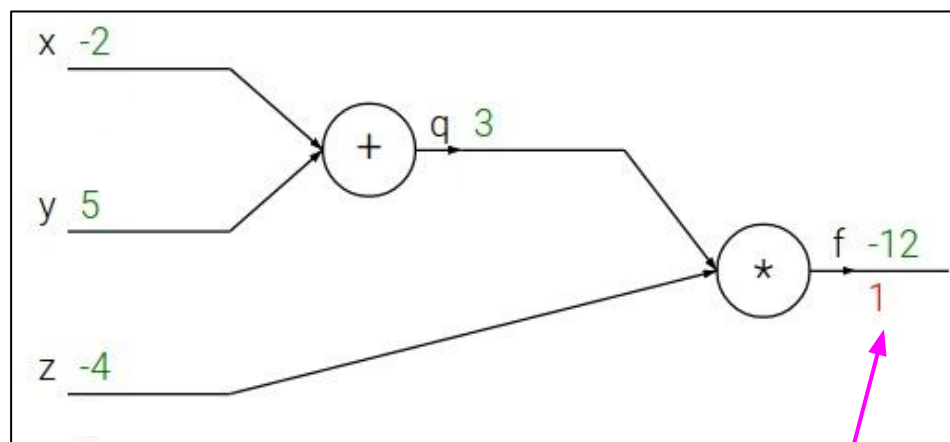
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial f}$$

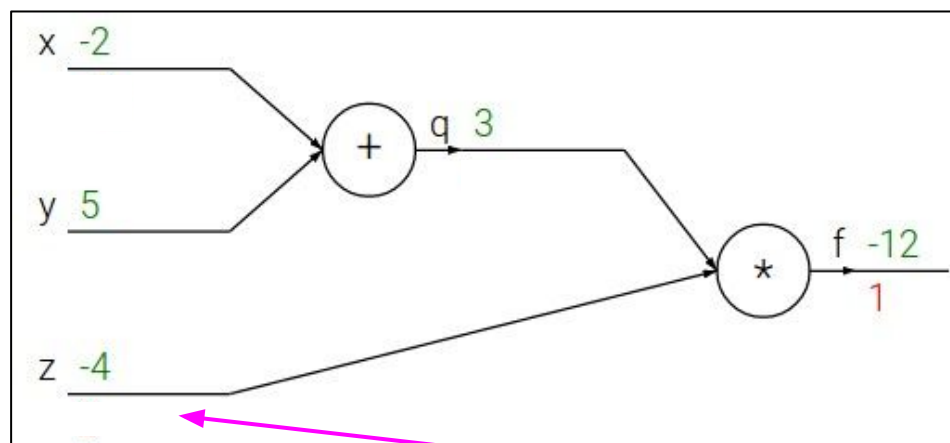
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial z}$$

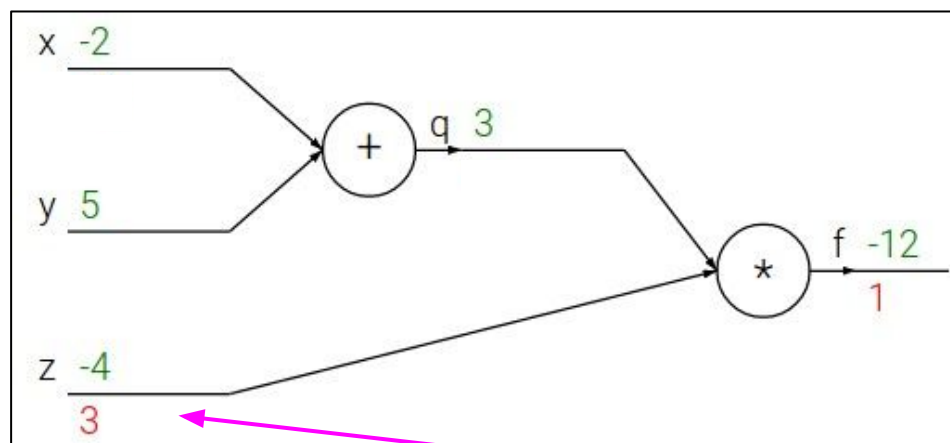
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial z}$$

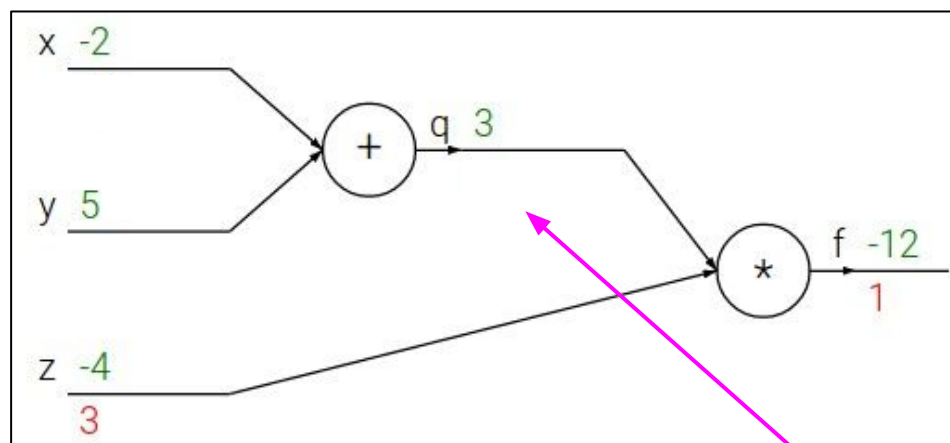
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial q}$$

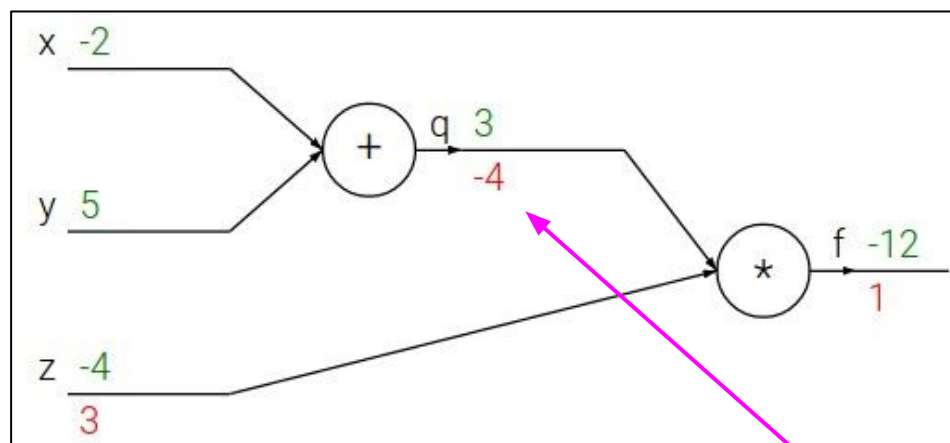
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial q}$$

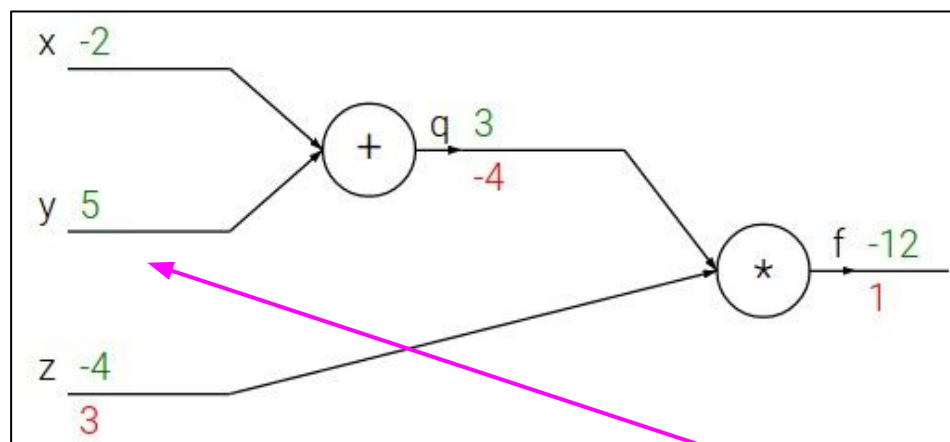
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial y}$$

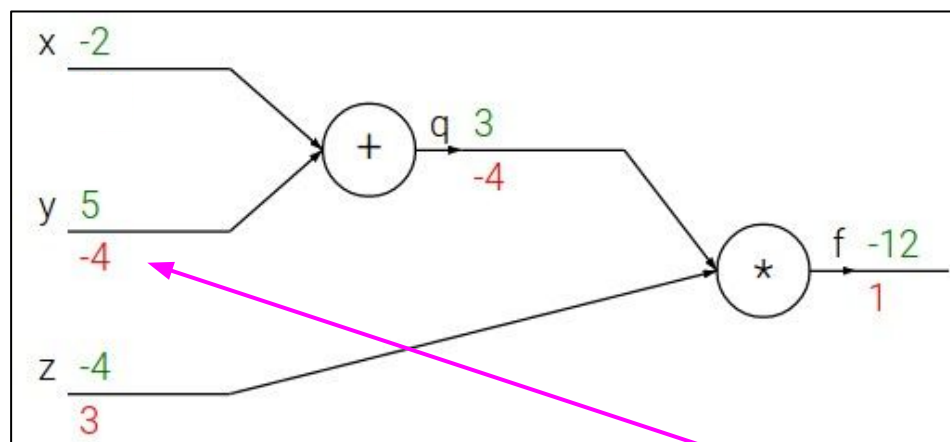
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



Chain rule:

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y}$$

$$\frac{\partial f}{\partial y}$$

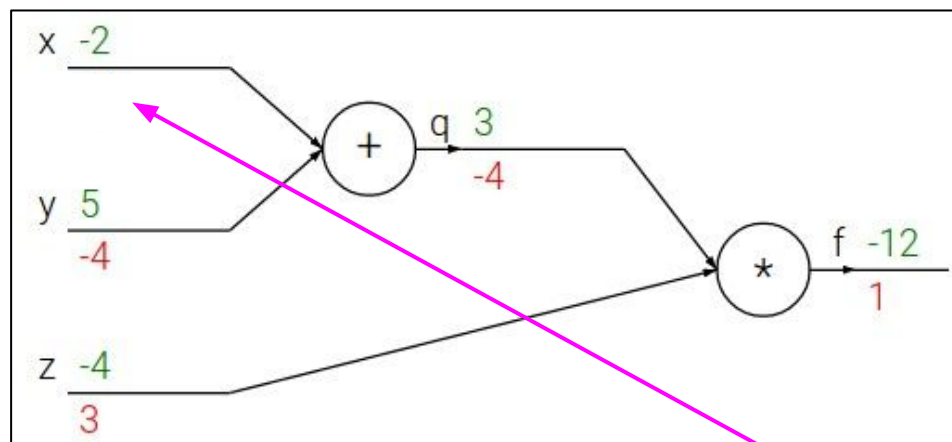
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial x}$$

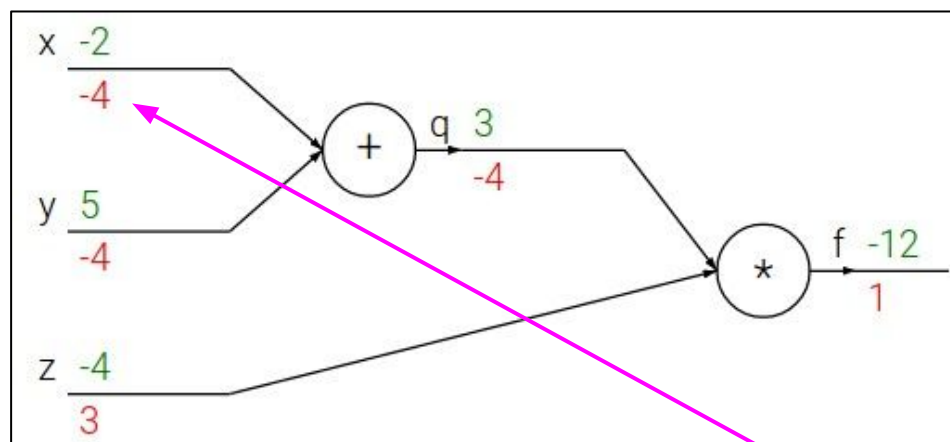
$$f(x, y, z) = (x + y)z$$

e.g. $x = -2, y = 5, z = -4$

$$q = x + y \quad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \quad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



$$\frac{\partial f}{\partial x}$$

Chain rule:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x}$$

